Semi-Autonomous Voice-Controlled Low-Cost Wheelchair for Indoor navigation

A Low-Speed Indoor Mobility System Built from a Manual Wheelchair

Assistive Robotics & Voice Navigation

A manual wheelchair is converted into a voice-controlled indoor vehicle that drives between named rooms using timed dead-reckoning routes, with no cameras or positioning sensors, for cost optimization.

Technical Report

Ilina Mukherjee

October 2026

Independent engineering project — personal build

Abstract

The project started with a simple question: can an ordinary manual wheelchair be converted into a voice-operated vehicle that carries a person between rooms without anyone pushing it, at a cost an ordinary household can afford? The target was a working system for under ₹50,000 (about US $500), built in multiple steps/stages that could each be tested on their own.

The first stage was mechanical. Two 24 V geared DC motors were fitted to the rear wheels through a chain-and-sprocket reduction. I sized the motors with a tractive-effort calculation and specified the sprocket bores and welded mounting plates, but the lathe and welding work was done by a mechanic at a local cycle workshop. The fitment was good but not precise: the chair drifted about 5° to the left over a 2 m straight run, and I corrected this in software by running the two wheels at slightly different speeds.

The navigation layer uses no cameras or positioning sensors. A Raspberry Pi turns a spoken room name into a list of timed movements, read from a directions file keyed by source and destination (for example bedroom-dining), and sends them one at a time to an Arduino Uno, which drives the motors and confirms each movement when it ends. A background thread listens for ‘stop’ and ‘resume’ at all times, and the chair keeps track of which room it is in, updating on arrival and accepting a correction by voice.

The finished chair cost ₹48,500 and moves between named rooms on spoken command, although it still needs supervision in daily use. Position is estimated only from elapsed time, so error grows with distance, and the chair cannot detect obstacles. The follow-up work covers graph-based route planning, a precision re-fit of the drivetrain, and closing the position loop with LiDAR or a camera and a Kalman filter.

1. Introduction

Many elderly people at home cannot walk on their own and depend on someone to push their wheelchair. A wheelchair that drives itself would help, but it costs too much for most families in India. In this project I converted an ordinary manual wheelchair so that it drives itself between rooms when the person in it says where to go.

Two motors drive the rear wheels, an Arduino controls them, and a Raspberry Pi listens for spoken commands. There are no cameras or position sensors. The chair works out how far it has gone from how long its motors have run. It can be stopped and resumed by voice at any moment, and it cost ₹48,500 (~USD 500).

2. Problem Statement

This project began with a personal observation rather than a technical question. When my grandfather lost the ability to walk because of a sudden knee complication, he became dependent on a manual wheelchair, and on whoever was available to push it. He weighed about 100 kg, so caregivers found it physically demanding to move him around the house, and he spent long hours confined to bed or sitting in one corner of the house. My grandmother is in a similar situation.

The goal of this project is to convert an ordinary manual wheelchair into a self-driving, voice operated vehicle that gives an elderly or mobility impaired person independent movement inside their own home. Commercial powered wheelchairs with any degree of autonomy are pricey for most middle class households in India, and none are built around voice as the primary interface. With my family's support, I therefore set a budget of under ₹50,000 (about US $500) and used a conventional manual wheelchair as the mechanical base.

The practical problem can be stated in terms of position. If the chair is in room S and the user says room T, the system has to move the chair from S to T with no sensor that reports where it is. Instead, the position is estimated from running time. At a fixed motor setting, the chair moves at a steady speed and turns at a steady rate.

- ✓ Distance ≈ $v_{cal} \cdot t$
- ✓ Turn angle ≈ $\omega_{cal} \cdot t$

Here v_{cal} and ω_{cal} are the linear speed and turn rate measured during calibration, and t is the running time of each movement.

In this project, a route is the fixed list of movements that takes the chair from one room to another. Each movement is a direction (straight, back, left or right) and the time it runs for, worked out from v_{cal} and ω_{cal}. Internally, the system keeps a table of routes, called the directions file, with one route for every start and destination pair. When the user names a room, the system looks up the route from the room the chair is in to the room named and runs it.

The project asks two questions. Are routes built from this model repeatable enough to bring the chair to the right room every time? And can the chair be stopped and restarted at any moment without losing its place in the route?

Alongside these questions, the system had to meet the following requirements.

- ✓ Operate indoors on level floors.
- ✓ No or minimal human oversight.
- ✓ Minimal configuration and simple day to day operation.
- ✓ Rated payload of up to 150 kg (occupant + accessories + chair).
- ✓ Understand spoken destination names within the house and travel to them unattended.
- ✓ Support an emergency stop and a resume from stop command by voice, without a physical switch.
- ✓ Tolerate unclear or accented speech to a reasonable degree.
- ✓ No dependency on cameras or active ranging sensors for the base cost tier (optional for those able to pay more).
- ✓ Guarantee a safe, controlled operating speed at all times.

3. Applications

The same pieces developed here, a retrofit drivetrain, voice control and timed point-to-point routes, are relevant wherever a person needs to move between a few fixed places without being pushed.

- **Home mobility:** Independent movement between rooms for elderly or mobility-impaired or people who doesn't has the ability to manoeuvre joysticks or push the rims themselves.
- **Reduced caregiver effort:** Removes the need to push a heavy occupant around the house for every move.
- **Low-cost retrofit:** Converts a wheelchair the household already owns instead of replacing it with a commercial powered unit.
- **Hands-free control:** Voice commands, including emergency stop and resume, for people who cannot operate a joystick or push rims.

4. Technical Challenges

Much of the design challenges came from problems encountered while building and testing the chair, rather than from a plan fixed in advance.

Challenge -1. Fitting a Drivetrain Without Precision Tooling

No factory fitting exists for this conversion, so the sprocket bores, motor mounting and frame reinforcement had to be made by hand. A mechanic at a local cycle workshop bored the sprockets on a manual lathe and welded the mounting plates to my specification and under my direction. I did not do this work myself. The result was about 85 to 90% fitment accuracy, and under a 70 kg load the chair drifted left by about 5° over a 2 m straight run.

Challenge -2. Sizing the Motors on a Tight Margin

Starting a 150 kg load on a 15° ramp needs about 570 W at breakaway, while the two motors are rated at 500 W continuous combined. The shortfall is accepted for two reasons. A DC motor delivers several times its rated torque for the short interval of starting, and the two wheels share the load. It is stated openly in this report rather than assumed away.

Challenge -3. Trimming Wheel Speed with 8-bit PWM

The Arduino's PWM has 256 levels, about 0.39% of full duty per step. Correcting the drift called for the left wheel to run 1.5% faster (duty 40 → 40.6), but the nearest available value is 41, which is a 2.5% difference. Straight line tracking therefore had to be finished by tuning on the real chair rather than by calculation.

Challenge -4. Navigating Without a Position Sensor

GPS does not work indoors, and cameras and ranging sensors were left out to hold down cost. Position can only be estimated from how long each movement has run at a known, steady speed. Every route therefore has to be calibrated by measurement, and the error grows with distance and with changes in battery voltage.

Challenge -5. Stopping the Chair Without Losing the Route

An emergency stop has to work while the Pi is busy waiting for the Arduino to finish a movement. A background listening thread handles that, but the Arduino confirms every command, including a stop, with the same DONE token. If the Pi reads the stop's DONE as the end of the movement it just interrupted, it skips to the next step instead of pausing. The interrupt flag is therefore checked again at the moment each DONE arrives.

Challenge -6. Knowing Where the Chair Is (Localization)

A route is only valid from the place where it was measured. After a route ends, the software must know which room the chair is in, and that belief must be correctable by voice when the chair has been moved by hand. Every route is therefore stored in the directions file under its start and destination room, so that any journey is a single, directly measured leg.

5. Technical Approach

The project began with the mechanical conversion and was built up in four stages, each tested before the next began: mechanical, electrical, embedded control and navigation software. A manual wheelchair carries two geared motors on its rear wheels, each with its own battery and motor driver. An Arduino Uno produces the motor signals, and a Raspberry Pi listens for speech and sends timed commands to the Arduino over a serial link. The full signal and power flow is shown in Figure 1.

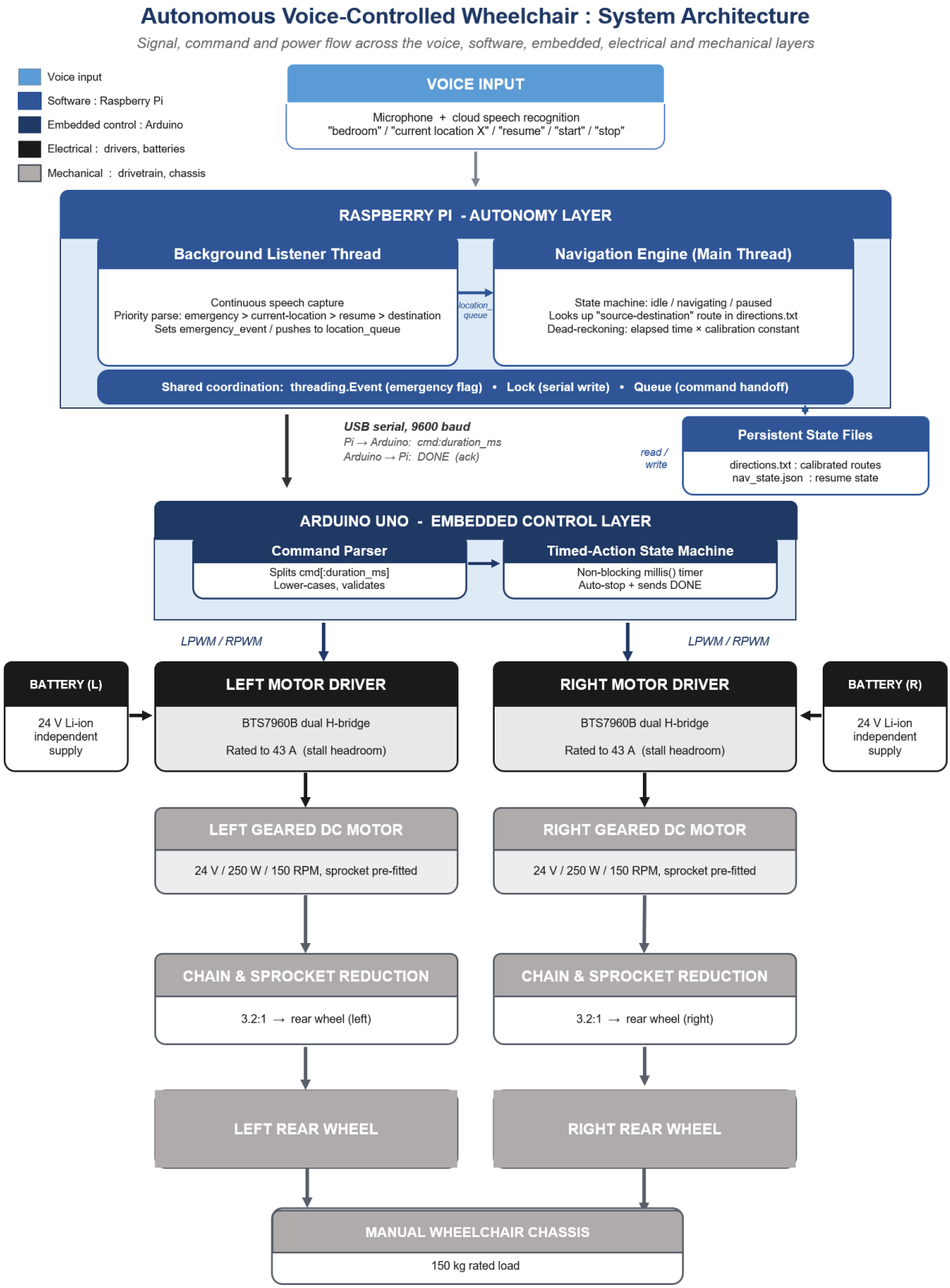


Figure 1. System architecture and signal flow.

5.1 Components and Cost

The build was costed against a ₹50,000 (≈US $500) target. The final bill of materials came to ₹48,500, with labour and incidental parts included:

Item	Qty	Approx. cost (₹)
Manual wheelchair (150 kg load-rated)	1	8,000
Geared DC motors, 24 V / 250 W / 150 RPM (sprocket pre-fitted)	2	5,000
Li-ion battery, 24 V	2	14,000
Motor drivers (BTS7960B) and speed-trim potentiometers	2 sets	1,000
Arduino Uno R3	1	3,000
Raspberry Pi 5.0	1	11,000
Sprockets, chains, 20 A-rated wiring, connectors, misc.	-	500
Workshop labour (sprocket machining and welding, carried out by a local cycle-shop mechanic, not the author)	-	6,000
Total		**48,500**

USD figures throughout this report use an approximate conversion of ₹96 per US $1 and are indicative, not contractual.

5.2 Mechanical Conversion

The mechanical base is an off-the-shelf manual wheelchair, chosen for structural margin (rated for the full 150 kg design payload) and for enough clearance between the rear wheel and the axle to seat a chain sprocket. No factory fitting exists for this conversion, so every mechanical interface, such as, the sprocket bore, the motor mounting and the frame reinforcement, had to be custom fitted at a local cycle workshop using a lathe rather than a calibrated machine shop.

I designed, select component, dimensioning and tolerancing. The physical fabrication, boring the sprockets to the axle diameter on the lathe and welding the mounting plates into the chair frame, was carried out by the mechanic to my specification and direction. The workshop had the lathe and welding equipment, and the training to use them, that a home build does not have access to. That constraint shaped the traction-system design described below.

Choosing the drive motors is a tractive effort problem. Tractive effort is the pushing force the driven wheels must apply on the floor to move the chair. Physically, it is the force at the point where the tyre touches the ground.

I worked it through for 150 kg (occupant, chair and electronics) at a target speed of 0.35 m/s, with an allowance for a 15° ramp and an acceleration of 0.5 m/s². Full working is in Appendix A. The question is how much power the motors need, and whether the motors I chose are enough.

The calculation runs in two steps. First, I find how hard the wheels must push on the floor, and how much power that takes. Then I check whether the motor's torque, after the chain reduction, can give

that push at the wheel. The table lists each formula in that order, with what each symbol means, and the value used for this chair.

Comparing the push at the wheel with the force needed for each floor and ramp gives the results below.

Steps	Quantity	Formula	What the symbols mean	Values for this chair
1	Force at breakaway	$F_{start} = m \cdot a + m \cdot g \cdot \sin\theta + \mu s \cdot m \cdot g \cdot \cos\theta$	m is the total mass, a the acceleration, g gravity, θ the ramp angle and μs the static friction coefficient. The three terms are the force to accelerate the chair, to hold it on the ramp, and to overcome static friction.	m = 150 kg, a = 0.5 m/s^2, g = 9.81 m/s^2, θ = 15°, μs = 0.6
2	Force at steady speed	$F_{roll} = m \cdot g \cdot \sin\theta + \mu r \cdot m \cdot g \cdot \cos\theta$	μr is the rolling friction coefficient. There is no acceleration term, since the speed is steady.	μr = 0.06
3	Power needed	$P_{start} = F_{start} \cdot v / \eta$ and $P_{roll} = F_{roll} \cdot v / \eta$	v is the speed and η the drivetrain efficiency, the share of motor power that reaches the floor.	v = 0.35 m/s, η = 0.8
4	Rated motor torque	$T_{motor} = P / \omega$, with $\omega = 2\pi \times N / 60$	P is the motor's rated power and N its rated speed in RPM. ω is the same speed in radians per second.	P = 250 W, N = 150 RPM
5	Torque at the wheel	$T_{wheel} = T_{motor} \times G \times \eta$	G is the chain reduction ratio, which multiplies the torque.	G = 3.2
6	Push at the wheel	$F_{wheel} = T_{wheel} / r$	r is the wheel radius.	r = 0.2921 m (23 in wheel)

Table 1. Parameters used for calculations (refer Appendices for details)

- **Power needed:** About 570 W at breakaway (static friction coefficient 0.6, 15° ramp, 0.5 m/s^2) and about 200 W, or 245 W with a 20% margin, to keep moving at steady speed up the 15° ramp (rolling coefficient 0.06). Drivetrain efficiency is taken as 0.8.
- **Motors selection:** Two 24 V, 250 W geared DC motors, 150 RPM rated, supplied with a drive sprocket already fitted. Together they are rated at 500 W continuous, which is about twice the steady state demand, but the breakaway figure is about 14% above it.
- **Breakaway value:** The breakaway figure is pessimistic. It combines the 15° ramp with a deliberately high static friction allowance and is evaluated at full speed. On a level floor the same calculation gives about 420 W, within the 500 W rating. Even this is an overestimate, since the tyres grip the floor rather than resist it, so the real starting load is smaller. A brushed DC motor can also supply well above its rated torque for the second or two it takes to start moving.
- **Torque:** Torque is the real limit. The chair moves slowly, so the motors are limited by torque, not power. Appendix A shows that their rated torque covers level floors and, at steady speed,

ramps up to about 1:12 (~5°) with room to spare, but not the 15° case. That case is a bounding one, and the chair is not intended for slopes that steep.

So, about 245 W is needed to keep the loaded chair moving even up a 15° ramp, and by power the 2 × 250 W motors are enough. Torque is what limits them, to level floors and gentle ramps, which is what the chair is designed for.

- **The chain and sprocket pair:** helps here. A 0.08 m sprocket on the wheel against the motor's 0.025 m sprocket gives a reduction of 3.2 to 1 (Appendix B). This steps the already geared 150 RPM motor output down to about 47 RPM at the wheel, before any electronic speed control. It also multiplies the motor's torque by about 3.2 at the wheel, which is what lets the motors push the chair up gentle ramps.

The harder problem was fitting the components without precision tooling. The workshop mechanic bored the sprocket on a manual lathe to match the axle diameter, and welded the motor housing, battery trays and driver electronics onto two handmade steel plates bolted into the chair frame. These were aligned by eye and by trial, rather than with a precision machine.

- **Fitment accuracy**: Roughly 85 to 90% by direct measurement, which is good enough to run, with a small, consistent sideways error.

- **Residual drift:** Under a 70 kg load, the chair drifts left by approximately 5° over a 2 m straight run.

- **Software correction:** The remaining error is absorbed by deliberately running the two wheels at slightly different speeds. This moves the tolerance budget from the mechanical side to the control side, a standard trade off when fabrication precision is the limiting resource.

5.3 Electrical System

Each wheel has its own independent power and drive chain: a 24 V lithium-ion battery, a BTS7960B dual H-bridge driver [9], and a DC motor-speed-controller potentiometer used for coarse trim.

- **Current draw:** At the motors' rated 250 W / 24 V, nominal current is approximately 10.4 A per motor (≈20.8 A combined).
- **Driver headroom:** The BTS7960B can handle up to about 43 A, far more than the 10.4 A each motor needs. I chose it with this extra room on purpose. When a DC motor starts from rest, it draws several times its normal current for a moment. This is because it has not yet built up back-EMF, the voltage a spinning motor makes against the supply. The wiring, rated at 20 A, is the real limit, so each motor can briefly take about 1.9 times its rated current.
- **Independent batteries:** Running each motor from its own battery, rather than a shared supply, stops one motor's startup transient from affecting the other's voltage rail. This matters because the two drivers are commanded to run at deliberately different duty cycles for straight-line correction (Appendix C).

Two independent battery and driver pairs cost slightly more than one shared system, but they give electrical isolation between the wheels. That is a reasonable choice because the whole steering strategy depends on controlling each wheel's speed independently and precisely.

5.4 The Chair as a Differential-Drive Rover

Connecting to foundational principles of robotics, converted chair is a differential-drive robot[4]. The two large rear wheels are driven independently, and the front castors only carry weight and swivel freely. There is no steering mechanism. The chair changes direction purely by running its two rear wheels at different speeds. This is the same model used for many small mobile robots [4], and it is what makes a timed-command scheme workable at all.

Let the left and right wheel speeds be v_L and v_R, and let L be the track width, the distance between the two wheel contact patches. The speed of the chair and its rate of turning are then

```
v = (v_L + v_R) / 2          ω = (v_R − v_L) / L
```

with ω positive anticlockwise when seen from above. The three cases that matter here, are:

- **Equal wheel speeds:** ω = 0 and the chair goes straight.
- **One wheel held (v_R ≈ 0):** the chair pivots about that wheel. Its centre moves on a circle of radius L/2 at a rate ω = v_L / L. This is how the chair turns right, and the left turn is the mirror image.
- **Equal and opposite speeds:** the chair would turn on the spot, because the centre does not move. It needs a reversing wheel, which the present drive does not use for turning.

Two practical consequences follow for this build.

- **A turn is not in place.** When the chair turns about one wheel, its centre still moves along the floor. In a quarter turn this distance is about (L/2)·√2, roughly 0.4 m for a track width of 0.56 m. The routes already include this, since the turn times were measured (calibrated) on the chair. This is one more reason the routes are measured rather than calculated.
- **Small speed differences matter.** If the wheels differ in speed by a fraction ε of the forward speed, the heading changes at ω = εv/L, and after a distance d the heading has turned through about ε·d/L radians. For ε = 2%, d = 2 m and L = 0.56 m that is 0.071 rad, or about 4°, which is the same order as the drift seen on the chair (Appendix C). So, a mismatch of a few per cent, easily produced by fitment and motor-to-motor variation, is enough.

The front castors add a further, less predictable effect. Until they have swung into line with the direction of travel they push sideways on the chair, and this probably contributes to the small kink seen at the start of a run and after a turn. It is not modelled here.

Speed also must be safe. At the operating speed of 0.245 m/s the kinetic energy of the 150 kg chair and passenger is ½mv² ≈ 4.5 J. If the chair is brought to rest at the same 0.5 m/s² used for acceleration in Appendix A, it stops within v²/(2a) ≈ 0.06 m. The stopping distance on the real chair depends on friction and load and has not been measured.

5.5 Embedded Control (Arduino)

The Arduino Uno sits between the Raspberry Pi and the two motor drivers. It handles everything that must happen in real time and cannot wait for a general-purpose operating system: generating the PWM signal for each motor, selecting forward or reverse through each driver's paired LPWM and RPWM inputs, and running a motor for a specific time (commanded) before reporting completion. It uses a non-blocking timer (milli second) rather than delays, so a new command such as a 'STOP' is still read while a movement is running.

The Uno's PWM is 8-bit (0–255), so the granularity in speed adjustment is one part in 255, about 0.39% of full duty. This is the practical limit on how precisely wheel speed can be controlled, to shape the straight-line correction:

Correction for the drift

- **Required correction:** the left wheel needs to run at roughly 1.5% higher duty than the right (duty 40 → 40.6) to cancel the observed drift.
- **Realisable correction:** 40.6 is not a representable 8-bit value, so it is rounded to 41, an actual differential of 2.5% rather than 1.5%. At this duty one step is 1/40 = 2.5% of the setting, so that is the smallest correction available.
- **Consequence:** this quantisation gap is why straight-line tracking is finished by tuning against the real chair rather than trusted to the arithmetic alone.

Turning uses the same differential principle taken further:

- **Right turn:** the left wheel runs at duty 40 (normal forward speed) while the right wheel is held at a low but non-zero duty of 5, for a calibrated duration.
- **Why not zero:** at zero duty the motor loses rotational control entirely and can be dragged into an uncommanded spin by the chair's own motion, especially under heavy load, which would make the turn angle unpredictable.
- **Why a lookup table:** the relation between hold time and turn angle is, in principle, the differential-drive result in Section 4.4 ($\omega = v_L / L$ when the other wheel is held). In practice it also depends on wheel slip under load, an uncertain track width and the castors, none of which the idealised formula captures. The formula is useful as a starting guess, but the final values are measured.
- **How it is established:** timed turns are run at known loads, and the resulting angle is recorded, giving a calibration table rather than a closed-form calculation. The same approach is used for straight-line distance in the navigation layer.

5.6 Navigation Software: Timed Dead Reckoning

The Raspberry Pi is the autonomy layer. It listens for speech, decides what the chair should do, and sends timed movement commands to the Arduino over a serial link, one at a time. The two devices use a deliberately minimal protocol: the Pi sends a command and a duration in milliseconds (for example `straight:11800`), and the Arduino runs it and, only when it finishes, replies with a single

acknowledgement token, `DONE`. Every higher-level behaviour, including resumable stops, step sequencing and state recovery, is built on that handshake, so it must be enforced strictly.

Indoors, GPS is not available, and cameras were left out to hold down cost, so dead reckoning is the only practical way to localise the chair. Position is inferred, never measured, from elapsed time at a known, constant speed. Each movement type (straight, back, left, right) always runs at one fixed, pre-calibrated duty cycle, so distance and elapsed time are directly proportional, and the chair's internal 'position' is simply a clock multiplied by a calibration constant (Appendix D).

Role of Direction File:

Every journey is described in a small directions file, keyed by its source and destination (for example `bedroom-dining`, not just `dining`), as a list of direction and duration pairs that ends in an explicit stop. The format is shown below, with durations in seconds. The values are illustrative; the quarter turn is set at 3.6 s because a pivot about one wheel at 0.245 m/s with a track width near 0.56 m takes about that long (Section 4.4).

```
bedroom-dining:
    straight:3
    right:3.6
    straight:6
    stop
```

The durations are not typed in as guesses. They come from two measurements:

- **Linear speed:** drive straight for a timed run, measure the distance, and compute v_cal = distance / time.
- **Turn rate:** turn for a timed run, measure the angle, and compute ω_cal = angle / time.
- **Route durations:** each duration is the required distance divided by v_cal, or the required angle divided by ω_cal. A change of motor, wheel or pulley only needs the two measurements repeated, not a rewrite of every route.

5.7 Interrupts, Stop and Resume

The chair has to respond to 'stop' at any moment, including while the Pi is waiting for the Arduino to finish a movement. Five design decisions make this reliable:

- **Acknowledgement-based flow control.** Rather than estimating how long a movement takes and sleeping for that time, which drifts out of sync the moment anything is slightly off, every command blocks until the Arduino confirms completion. This ties the Pi's model of the chair's position to what the motors actually did.
- **A background listening thread.** The system listens for speech at all times using cloud speech recognition [10], on a thread separate from the one executing the route. The two threads coordinate through a shared interrupt flag and a lock around the serial connection, so they never write to the Arduino at the same instant.

- **Disambiguated acknowledgements.** The Arduino acknowledges every command, including an emergency stop, with the same DONE token, so a stop's acknowledgement can arrive while the route thread is still waiting for the movement it just interrupted. The system checks the interrupt flag at the instant each acknowledgement is received, not only before it starts to wait. A DONE that cannot be attributed with confidence to the command in flight is treated as an interruption, not a completion.
- **A strict resume-only policy.** While a route is paused, the system accepts exactly one voice command: 'resume'. A destination name or a 'current location' correction spoken during a pause, whether deliberate or through misrecognition, is ignored, so an emergency stop cannot be silently overridden. On resume, only the remaining time of the interrupted step is run (Appendix D).
- **Crash- and restart-safe state.** The active source, destination, current step and any remaining duration are written to a small, persistent file after every step. A stop and resume cycle, or even a full Raspberry Pi restart, recovers exactly where the chair stopped, without re-running or skipping any part of the route.

5.8 Location Tracking and Point-to-Point Routes

A route is only valid from the place where it was measured, so the software always keeps a belief about which room the chair is in:

- **Source-destination pairs.** Every route is keyed by its exact source and destination, not by destination alone. `bedroom-dining` and `dining-bedroom` are two separate, independently measured entries, not assumed to be mirror images of each other. Any journey is therefore one directly calibrated leg, never a composition of several routed through a shared reference point. In graph terms this is a point-to-point (mesh) topology rather than a hub-and-spoke one.
- **Voice-correctable location. The Role of "Where I am currently"?** The belief about the current room updates automatically when a route completes and can be corrected at any time by saying 'current location' followed by a room name. The command is accepted only while the chair is idle, so a correction can never collide with a route whose state is still in flight. This remains open-loop dead reckoning, since nothing senses the chair's true position, but it stops positional error compounding silently across a chain of hops.
- **Example.** Saying 'current location bedroom' and then 'dining' runs the `bedroom-dining` route. On arrival the location becomes dining, so saying 'drawing' next runs `dining-drawing`.
- **Rules.** Naming the room the chair is already in does nothing. If the "current location" is not yet known, the request is refused and the user is asked to set it first. If a pair has no entry in the directions file, the missing route is reported.
- **Calibration cost.** Because every pair is stored independently, full connectivity across N rooms needs up to N×(N−1) entries. This is the accepted cost of keeping every journey to a single measured leg (Section 5.3).

5.9 Processing Procedure

The whole procedure is summarised below as pseudocode. It covers the Pi's listener thread and main thread, and the Arduino.

```
BOOT:
    Load nav_state.json (current location, active route, step, remaining time)
    Start the background listener thread

LISTENER THREAD (always running):
    Capture speech and recognise the text
    IF text contains "start" or "stop":
        Set the emergency flag
        Send the command to the Arduino immediately
    ELSE IF text contains "current location <room>":
        Queue a location update
    ELSE IF text contains "resume":
        Queue a resume request
    ELSE IF text contains a known room name:
        Queue a destination request

MAIN THREAD:
    Take the next item from the queue
    IF resume AND a route is paused:
        Continue from the saved step with the remaining time
    IF location update AND the chair is idle:
        Store the new current location
    IF destination AND the chair is idle:
        IF current location is unknown: ask for it and stop
        IF destination equals current location: do nothing
        Look up "current-destination" in directions.txt
        FOR each step (direction, duration):
            Send "direction:duration_ms" to the Arduino
            Wait for DONE (abort if the emergency flag is set)
            Save progress to nav_state.json
        Set current location to the destination

ARDUINO:
    Parse "direction[:duration_ms]"
    Drive the motors at the fixed duty cycle for that direction
    When the timer expires, stop and reply DONE

OUTPUT:
    Timed, acknowledged movement of the chair from room to room
```

6. Experiment Design, Results, and Demos

6.1 Experiment Design

The chair was tested in stages that follow the build. Each test used the chair itself rather than a simulation:

- **Straight-line test:** the chair was driven over a 2 m straight run under a 70 kg load, and the sideways drift was measured as an angle.

- **Wheel-speed check:** wheel rotation at the operating duty (about 40/255) was observed and compared with the value predicted from the motor RPM and the reduction ratio (Appendix B).
- **Turn calibration:** timed turns were run at known loads and the resulting angle was recorded, to build the duration-to-angle table.
- **Route and interrupt tests:** multi-step routes between named rooms were run from the directions file, with stop and resume commands issued during movement, to check that the chair paused, resumed with the remaining time of the step, and continued to the destination.

Calibration runs. Three sets of calibration measurements were done on the same floor at home, which is about 60% marble and 40% tiles. Each value is the average of several runs (5 runs).

Measurement	How it was done	Load
Straight runs	Drive straight for 1 to 5 wheel rotations. Record the time and measure the distance covered.	70 kg
90° turns	Turn 90° and record the time.	70 kg
Drift	Drive straight for 1, 2,3 and 5 rotations. Measure the angle of drift to the left, before and after the electrical adjustment (left wheel duty 40 → 41).	70 kg

Table 2. Calibration runs

6.2 Results and Observations

Table 2 compares the figures expected from calculation, or set as targets, with what was observed or realised on the chair during different stages of the build phase and calibration.

Quantity	Expected or Targets (refer Appendices)	Observed during build phase different stages	Observed – Calibration Runs with 70KG load
Wheel speed at duty ≈ 40/255	≈ 7.4 RPM	≈ 8 RPM	Average 7 RPM (one rotation in 8.57 s)
Wheel linear speed	≤ 0.35 m/s (design ceiling)	≈ 0.245 m/s	0.19 to 0.22 m/s
90° turns	3.6 s	3.6 s	4.0 s on an average
Straight-line drift (70 kg, 2 m)	0°	≈ 5° left, before software correction	5°, 5.1° and 5.4° left at 1, 2 and 5 rotations
Left-wheel duty correction	+1.5% (40 → 40.6)	+2.5% (40 → 41)	Not much deviation noticed at 1 and 2 rotations. Slight deviation to the right at 5 rotations (not measured)
Total build cost	₹50,000 target	₹48,500	₹48,500

Table 3. Expected versus Observed figures.

- The relation between PWM duty and wheel speed is close to linear at the operating point, which is why open-loop speed control was usable without feedback.

- The drift has a consistent direction under a given load, so a fixed duty correction compensates for it, although 8-bit quantisation means the correction overshoots and final tuning is empirical.
- Acknowledgement-based sequencing keeps the Pi's view of the route tied to what the motors did, and a resumed step runs only its remaining time rather than restarting.
- Route-level accuracy (how close the chair stops to the intended spot over repeated runs) other than rear wheel alignment, also depends on battery state, tyre pressures and load, and is one of the first measurements planned in Section 6.2.

Expected against Observed: There have been some deviations between what was expected and what was observed during calibration runs or otherwise

- **The linear speed, is lower than the expected**

  The likely reasons are the friction grows with the weight on the wheels, the precision of marking the stopping point and reading the tape, and the tyres flattening slightly under 70 kg, which changes the rolling radius. The floor also changes between marble and tiles along the run.
- **The 90° turning time was higher at 4.0 s (which is 0.4 s i.e. 11% longer)**.

  This friction grows with the weight on the wheels, so with 70 kg the motors have to give more torque. The motors run open loop at a fixed PWM duty, and a DC motor slows down when it has to give more torque, so the wheels turn more slowly and the turn takes longer. The inner wheel, held at only duty 5, is the most likely to slow down or stop under load. The person's weight is also not shared exactly evenly between the two wheels, so the two wheels slow down by different amounts.
- **There has been a drift of 5° left**

  It is a wheel alignment problem from the hand fitted drivetrain, the same kind of problem as a car whose wheels are out of alignment. If the rear axle is not exactly square to the frame, the chair rolls straight but at a small, fixed angle to the way it is facing, so the drift angle stays about the same at every distance. The small rise from 5.0° to 5.4° could come from uneven load. A person rarely sits exactly in the middle, so one wheel carries more weight, meets more rolling friction and turns slightly slower. The one step PWM change adds a slight curve that cancels the fixed angle over short runs and goes slightly too far over long runs, which matches the slight deviation to the right at 5 rotations.

6.3 Comparing Route Organisations

Two ways of organising the routes were considered. In a hub-and-spoke layout every journey runs through one fixed home position. In the point-to-point layout used here, every source-destination pair has its own measured route.

	Hub-and-spoke (always via home)	Point-to-point (pairs)
Routes to calibrate, N rooms	2(N−1)	N(N−1)
Example: 3 rooms / 6 rooms	4 / 10	6 / 30
Journey between two non-home rooms	Two legs through home	One directly measured leg
Position error over a journey	Adds across legs	Single leg only
Starting room must be known	No, always home	Yes, needed for the lookup

Table 4. Hub-and-spoke versus point-to-point routes.

The point-to-point layout was chosen. It trades a larger route table for single-leg journeys, and the size of that table can be reduced later with graph-based planning (Section 7.2).

6.4 Current Limitations

- **Slope capability:** the rated motor torque covers level floors and gentle ramps (~ 5°), but not a 15° ramp, and the driver and wiring limit the short-term overload to roughly twice the rated current (Appendix A). The chair is intended for level indoor floors and gentle ramps only.
- **Mechanical alignment:** workshop fitment without machine-shop precision leaves a residual straight-line drift (about 5° over 2 m before software correction) and only 85–90% fitment accuracy. Software correction mitigates this but does not remove it.
- **Open-loop dead reckoning:** there are no wheel encoders, IMU or absolute position reference. Position is inferred purely from elapsed time at a calibrated speed, so drift accumulates with distance, time and battery-voltage variation, and is compounded by the mechanical alignment error.
- **No obstacle detection:** cameras and ranging sensors were excluded by design to hold the base cost down, so the chair cannot detect or avoid an obstacle in its path.
- **Unverified location belief:** pairwise calibration (Section 5.8) stops error compounding across hops, but the chair's belief about where it is comes only from its own bookkeeping. An incorrect 'current location' statement or a compounded deviation from desired path is accepted with no independent way to catch the mistake.
- **Route-table growth:** because every source-destination pair is calibrated independently, full connectivity for N rooms needs up to N×(N−1) entries, so calibration effort grows faster than the number of rooms.
- **Supervision:** the chair still needs periodic human supervision in real-world use.
- **PWM resolution:** 8-bit PWM limits speed trimming to steps of about 0.39%, so fine corrections are always subject to rounding (which makes it 2.5%, instead of desired 1.5%)

- **Speech recognition:** it depends on a cloud service, so it needs network connectivity and is subject to its latency and availability. Accent adaptation, though an original design goal, is not yet separately engineered.
- **No safety certification:** this is a self-built device carrying a person and has not been through any independent safety assessment.

6.5 Demos

Video demonstrations of the following runs accompany this report:

- **Route run:** bedroom to dining room on a spoken room name, from a known starting location.
- **Emergency stop and resume:** a ‘stop’ during a straight leg, followed by ‘resume’, continuing with the remaining time of the step.
- **Ignored commands while paused:** a destination name spoken during a pause, showing that only ‘resume’ is accepted.
- **Location correction:** ‘current location’ spoken after the chair has been moved by hand, followed by a route from the corrected room.

7. Related Work and Follow-Up Work

7.1 Related Work

Powered and ‘smart’ wheelchairs have been studied for decades. Reviews such as Leaman and La [1] cover systems that add sensors, shared control and alternative user interfaces to a standard wheelchair, and Simpson and Levine [2] demonstrated voice control of a powered wheelchair. Many of these systems rely on ranging sensors or cameras for obstacle avoidance. This project instead leaves sensing out and uses voice as the main interface.

Position estimation for mobile robots usually combines odometry, which is dead reckoning from wheel motion, with external sensing. Borenstein, Everett and Feng [3] describe the sources of odometry error that make unaided dead reckoning drift, and Siegwart and Nourbakhsh [4] give the differential-drive kinematics used in Appendix C. Probabilistic localisation based on the Kalman filter [5, 6] corrects this drift with independent measurements.

Planning routes over a set of places is a classical graph problem. Dijkstra’s algorithm [7] finds shortest paths from one node, and Floyd’s algorithm [8] finds them for all pairs at once. Appendix F describe how these could replace hand-measured routes between every pair of rooms.

The approach in this project is considerably simpler than the systems above: it uses no position sensor and relies on routes that are measured once and run by time. That keeps the cost low, at the price of the accumulating error those systems are designed to remove.

7.2 Follow-Up Work

The next steps address the three limits that matter most: the lack of any independent position reference, the size of the route table and the mechanical imprecision.

Closing the dead-reckoning loop

- **Closed-loop position correction:** fusing dead reckoning with an independent reference, such as LiDAR scan matching or visual landmarks from a camera, through a Kalman filter, so accumulated drift is corrected rather than only predicted (Appendix E).
- **Wheel encoders:** measuring wheel rotation directly instead of inferring distance from time at an assumed speed, which removes the largest source of silent drift (battery sag, load variation).
- **Low-cost intermediate step:** simple IR threshold sensors or markers at doorways and room entries to re-anchor the position estimate before a full LiDAR or vision stack is added.
- **Location sanity check:** comparing the time-implied position against a simple independent cue, so an incorrect 'current location' statement or a missed arrival can be flagged rather than trusted.
- **Offline speech recognition:** to remove the network dependency and work toward the original accent-robustness goal.
- **Systematic measurement:** repeated runs of each route at different battery levels, tyre pressures and loads, recording how far the chair stops from the intended spot.

The sensing and compute upgrade needed for closed-loop correction (LiDAR or vision hardware, plus the processing to use it) is expected to push the total build cost beyond ₹60,000 (≈US $600+), above the original ₹50,000 target. That is a deliberate trade-off between cost and robustness, to be taken on in a second cost tier rather than in the base build. The graph-planning and mechanical-precision items are comparatively cheap, since they are mostly software and one-time fabrication effort.

Graph-based route planning

The point-to-point layout is, formally, a complete directed graph: every room is a node, and every calibrated entry is a directed, time-weighted edge. Treating it explicitly as a graph is the natural way to curb its N×(N−1) growth (Appendix F):

- **Sparse calibration:** only room pairs with a clean, obstacle-free path between them, typically adjacent rooms, are calibrated and stored as edges.
- **Shortest-path planning:** a destination with no direct edge is reached by computing a path through the known edges (Dijkstra's algorithm, or breadth-first search if every edge is weighted equally).

Mechanical and electrical precision

- **Finer speed trimming:** the Uno's 16-bit timer (on pins 9 and 10) allows PWM at a far finer resolution than the default 8 bits, which would let a 1.5% correction be realised as 1.5% instead of being rounded to 2.5%.

- **Precision re-fit:** a professionally machined, jig-aligned sprocket and motor mount, in place of the by-eye fitting described in Section 4.2, would remove the residual 5° drift at its mechanical source instead of compensating for it in software.

8. Conclusion

The system meets its first-level brief: a manual wheelchair, converted with the help of a local workshop, drives itself between named indoor destinations on spoken command, can be stopped and resumed by voice, and was built for ₹48,500, inside the original target. It is still open-loop, sensor-light and in need of a careful eye some of the time. My grandmother uses it today; my grandfather, for whom it was first imagined, did not live to see it finished. Closing the loop on position, so that the chair knows rather than estimates where it is, is the clear next step and the one most worth the additional cost.

9. Gallery

Photographs to accompany this report:

- **Converted wheelchair:** the finished chair with motors, batteries and electronics fitted.
- **Drive mounting:** close-up of the motor, sprocket and chain on a rear wheel.
- **Workshop fabrication:** sprocket boring and the welded mounting plates, made at the cycle workshop.
- **Electronics:** Raspberry Pi, Arduino Uno and the two motor drivers.
- **Route test:** the chair travelling between rooms on a spoken command.

10. References

[1] Leaman, J. and La, H. M. A Comprehensive Review of Smart Wheelchairs: Past, Present and Future. arXiv:1704.04697, 2017.

[2] Simpson, R. C. and Levine, S. P. Voice control of a powered wheelchair. IEEE Transactions on Neural Systems and Rehabilitation Engineering, 10(2), 2002.

[3] Borenstein, J., Everett, H. R. and Feng, L. Where Am I? Sensors and Methods for Mobile Robot Positioning. University of Michigan, 1996.

[4] Siegwart, R. and Nourbakhsh, I. R. Introduction to Autonomous Mobile Robots. MIT Press, 2004.

[5] Thrun, S., Burgard, W. and Fox, D. Probabilistic Robotics. MIT Press, 2005.

[6] Kalman, R. E. A new approach to linear filtering and prediction problems. Journal of Basic Engineering, 82(1), 1960.

[7] Dijkstra, E. W. A note on two problems in connexion with graphs. Numerische Mathematik, 1, 1959.

[8] Floyd, R. W. Algorithm 97: Shortest path. Communications of the ACM, 5(6), 1962.

[9] Infineon Technologies. BTS7960B High Current PN Half Bridge datasheet.

Appendix A - Traction Power Sizing

Design assumptions used to size the drive motors:

Quantity	Symbol	Value
Total mass (occupant + chair + electronics)	m	150 kg
Target top speed	v	0.35 m/s
Target acceleration	a	0.5 m/s²
Static friction coefficient (breakaway)	μs	0.6
Rolling friction coefficient (sustained)	μr	0.06
Assumed worst-case ramp angle	θ	15°
Drivetrain efficiency	η	0.8
Gravitational acceleration	g	9.81 m/s²

Breakaway tractive force and power (acceleration + slope + static friction):

$$F_{start} = m \cdot a + m \cdot g \cdot \sin\theta + \mu s \cdot m \cdot g \cdot \cos\theta$$
$$= (150)(0.5) + (150)(9.81)(\sin 15°) + (0.6)(150)(9.81)(\cos 15°)$$
$$= 75 + 380.8 + 852.9 \approx 1308.7 \text{ N}$$
$$P_{start} = F_{start} \cdot v / \eta = (1308.7)(0.35) / 0.8 \approx 572.5 \text{ W}$$

Sustained rolling force and power (steady speed, rolling friction, no acceleration):

$$F_{roll} = m \cdot g \cdot \sin\theta + \mu r \cdot m \cdot g \cdot \cos\theta$$
$$= 380.8 + (0.06)(150)(9.81)(\cos 15°) \approx 466.1 \text{ N}$$
$$P_{roll} = F_{roll} \cdot v / \eta = (466.1)(0.35) / 0.8 \approx 203.9 \text{ W}$$

Adding a 20% margin to the sustained figure gives 203.9 × 1.2 ≈ 245 W, about half of the 500 W combined continuous rating of the motors. The breakaway figure of 572.5 W is about 14% above that rating. It is a bounding case: the static-friction coefficient of 0.6 is a deliberately high allowance rather than a measured rolling resistance, and the power is evaluated at the full 0.35 m/s even though breakaway happens at almost zero speed. On a level floor (θ = 0) the same calculation gives F_start = 75 + 0.6 × 1471.5 ≈ 957.9 N and P_start ≈ 419 W, which is within the rating.

Power is the wrong quantity to stop at, though. The chair moves slowly (about 8 RPM at the wheel), so the motors are limited by torque, and a torque check is needed.

Rated motor torque, and the thrust it gives at the wheels through the 3.2:1 reduction (wheel radius 0.2921 m from the 23 in wheel, efficiency 0.8):

$$T_{motor} = P / \omega = 250 / (2\pi \times 150 / 60) = 250 / 15.71 \approx 15.9 \text{ N} \cdot \text{m}$$
$$T_{wheel} = 15.9 \times 3.2 \times 0.8 \approx 40.7 \text{ N} \cdot \text{m per wheel}$$
$$F_{wheel} = 40.7 / 0.2921 \approx 139.5 \text{ N per wheel} \rightarrow \approx 279 \text{ N for both wheels}$$

The thrust needed in several cases, using $F = m \cdot a + m \cdot g \cdot \sin\theta + \mu \cdot m \cdot g \cdot \cos\theta$ with $m \cdot g = 1471.5$ N, compared with the 279 N available at rated torque:

Case	Required thrust (N)	Share of rated thrust
Level floor, steady (μr = 0.06)	88	32%
Level floor, accelerating at 0.5 m/s²	163	59%
1:12 ramp (4.8°), steady	210	75%
15° ramp, steady	466	167%
15° ramp, breakaway (μs = 0.6, 0.5 m/s²)	1309	469%

Motor torque is roughly proportional to current. The 20 A wiring therefore limits the short-term overload to about 20 / 10.4 ≈ 1.9 times rated torque, or about 536 N of thrust. That is enough to start on a level floor and to climb a 1:12 ramp including acceleration (about 285 N), but it cannot supply the breakaway case on a 15° ramp. The 15° figure is therefore a deliberately severe bound, and the chair is intended for level floors and gentle ramps. This is stated as a limitation in Section 5.4.

Appendix B - Mechanical Reduction and Wheel Speed

Chain-and-sprocket reduction ratio from the measured sprocket diameters:

```
ratio = d_wheel_sprocket / d_motor_sprocket = 0.08 / 0.025 = 3.2 : 1
```

The exact ratio is the ratio of tooth counts, which should be confirmed by counting teeth, since measured outer diameters differ slightly from pitch diameters. Wheel speed at the rated motor speed:

```
Wheel RPM = Motor RPM / ratio = 150 / 3.2 ≈ 46.9 RPM
```

Wheel linear speed for a 23 in (0.584 m) diameter wheel, with circumference $\pi \times 0.584 \approx 1.835$ m:

```
v = (RPM / 60) × circumference
At 46.9 RPM:  v ≈ 1.43 m/s   (unloaded mechanical top speed)
At ≈ 8 RPM (duty ≈ 40/255, the operating setting):  v ≈ 0.245 m/s
```

0.245 m/s is comfortably under the 0.35 m/s design ceiling of Appendix A, so the operating point is conservative. The expected speed at a duty of 40/255, taking speed as proportional to duty, is

```
(40 / 255) × 46.9 RPM ≈ 7.4 RPM
```

which is close to the ≈ 8 RPM observed. A DC motor under light, nearly constant load is close to linear in speed against duty cycle, and that is why open-loop PWM speed control was usable without feedback.

Appendix C - Differential-Drive Kinematics and PWM Resolution

Wheel speeds to chair motion. For wheel speeds v_L and v_R and a track width L, the chair's forward speed v and turn rate ω (positive anticlockwise) are [4]:

```
v = (v_L + v_R) / 2
ω = (v_R − v_L) / L
```

When ω ≠ 0 the chair moves on a circle about an instantaneous centre, and the centre of the chair is at a distance R from it:

```
R = (L / 2) · (v_R + v_L) / (v_R − v_L)
```

With the right wheel held (v_R = 0) this gives |R| = L/2, a pivot about the stationary wheel, with ω = −v_L / L (clockwise, a right turn). For v_L = v_R, R is infinite and the chair goes straight.

Between two instants a short time dt apart, the position (x, y) and heading θ of the chair's centre change by

```
dx = v cosθ dt        dy = v sinθ dt        dθ = ω dt
```

Because every step in a route is either straight (ω = 0) or a turn at a fixed rate, these integrate in closed form. A straight step of duration t moves the chair a distance v_cal · t along its heading. A turn of duration t changes the heading by ω_cal · t. This is what the dead-reckoning arithmetic in Appendix D relies on.

Turn time. For a pivot about the stationary wheel, a turn of Δθ takes

```
t = Δθ · L / v_L
```

For a quarter turn (Δθ = π/2), v_L = 0.245 m/s where L = 0.56 m, t = 1.571 × 0.56 / 0.245 ≈ 3.6 s. The value of L was measured to 0.56 m and is a standard across most manual wheelchairs. During the turn the chair's centre moves along an arc of radius L/2 = 0.28 m, a chord of about 0.28 × √2 ≈ 0.40 m. In practice slip and the castors change both numbers, so the turn is timed on the chair and the formula serves only as a starting point.

Straight-line drift. Suppose the right wheel runs faster than the left by a fraction ε, so v_R = v_L (1 + ε). Then ω = ε v_L / L, and over a distance d travelled in a time d/v the heading changes by about

```
Δθ ≈ ε · d / L
```

For ε = 2%, d = 2 m and L = 0.56 m, Δθ ≈ 0.071 rad ≈ 4°. The path direction seen over the 2 m is about half of this heading change, so the observed figure of about 5° corresponds to a mismatch of the order of a few per cent. Because the sign of the mismatch is fixed by the hardware, it is cancelled by trimming the duty of one wheel.

PWM resolution. 8-bit PWM has 256 duty levels (0 to 255), so one step is

```
1 / 255 ≈ 0.392% of full duty
```

At the operating duty of 40 the relative step is larger, 1/40 = 2.5% of the setting. The right-wheel correction needed for the observed drift is

```
duty_right = duty_left × 1.015 = 40 × 1.015 = 40.6
```

40.6 cannot be represented, and the nearest value is 41:

```
actual differential = (41 − 40) / 40 = 2.5%   (against 1.5% required)
```

So the realisable correction overshoots by one percentage point. Final tuning is therefore done on the chair, and the 16-bit timer of the Uno could remove most of this rounding (Section 6.2).

Appendix D - Dead-Reckoning Calibration and Error Growth

For a fixed, known commanded duty cycle, the calibration constants come from two short physical test runs:

```
Linear speed:   v_cal = measured_distance / measured_time
Angular rate:   ω_cal = measured_angle / measured_time
```

A route is then generated as a sequence of durations:

```
straight / back duration = required_distance / v_cal
left / right duration    = required_angle / ω_cal
```

Because v_cal and ω_cal depend only on the motor, wheel and PWM configuration and not on the route, a single recalibration of two short test runs is enough after any hardware change (new wheels, new pulleys, a different duty cycle), without re-deriving every route by hand.

Position estimate while executing a step of planned duration T_planned, interrupted after elapsed time t_elapsed:

```
remaining_duration        = T_planned − t_elapsed
distance_already_covered = v_cal × t_elapsed
```

This is the basis of the resumable-stop behaviour in Section 4.7: because speed is held constant and known within any one step, elapsed time is a direct and reliable proxy for the distance covered in that step.

How errors grow. Dead reckoning has no way of correcting itself, so every small error carries through to the end of a leg. Three sensitivities are worth having in hand. They use illustrative figures, not measurements.

- **Speed error along the track.** The distance covered is $d = v \cdot t$, so a fractional speed error $\delta v/v$ gives the same fractional error in distance. A 5% sag in speed over a 6 m leg leaves the chair 0.3 m short.
- **Heading error across the track.** A heading error $\delta\theta$ at the start of a straight leg of length d produces a sideways error of $d \cdot \sin \delta\theta$ at the end. A 3° error over 6 m gives $6 \times \sin 3° \approx 0.31$ m, and the observed 5° drift over 2 m would, uncorrected, give about 0.17 m.
- **Turn timing.** A turn of duration t at rate ω has an angle error of $\omega \cdot \delta t$ for a timing error δt. The Arduino timer works in milliseconds, so timing itself contributes very little. The larger turn errors come from changes in speed and slip, which alter ω itself.

These errors add over the legs of a route. That is the reason the routes in this project are measured as single legs between pairs of rooms (Section 4.8) and the location belief is reset on every arrival.

Appendix E - Closed-Loop Correction (Follow-Up Work, Concept Only)

A Kalman filter [6] maintains a running position estimate $\hat{x}$ as a weighted blend of a motion-model prediction and an independent measurement, weighted by their relative uncertainty:

$$\text{Prediction (dead reckoning):} \quad \hat{x}^- = \hat{x} + v_{cal} \cdot \Delta t$$
$$\text{Correction (on a new fix } z\text{):} \quad \hat{x} = \hat{x}^- + K(z - \hat{x}^-)$$

The Kalman gain K is derived from the relative uncertainty (variance) of the dead-reckoning prediction and of the new measurement, giving more weight to whichever source is currently more trustworthy. A new fix z would come from LiDAR scan matching or from a recognised visual landmark at a known position. In one dimension the same idea can be written with a variance P for the estimate, process noise Q for each dead-reckoning step and measurement noise R for the fix:

$$P^- = P + Q$$
$$K = P^- / (P^- + R)$$
$$P = (1 - K) \cdot P^-$$

Between fixes P grows by Q at every step, which is the drift described in Appendix D, and each fix pulls it back down. Replacing dead reckoning's open-loop, ever-growing uncertainty with this kind of periodic correction is the most valuable upgrade identified for the next build stage, and the main cost driver pushing the project beyond its original budget.

Appendix F - Graph-Based Route Planning (Follow-Up Work, Concept Only)

The point-to-point routing of Section 4.8 can be formalised as a directed, weighted graph G = (V, E). Each room is a vertex $v \in V$, and each calibrated source-destination entry in the directions file is a directed edge (u, v) with weight w(u, v) equal to that leg's total commanded duration. At present E is the complete set of ordered room pairs, $|E|$ up to $|V| \cdot (|V| - 1)$, because every entry is calibrated by hand. The proposed change is to populate E only with physically direct, cleanly driveable legs (typically adjacent rooms) and compute everything else.

Shortest path between a known current location s and a requested destination t, by edge relaxation, the core step of Dijkstra's algorithm [7]:

```
dist[s] = 0;  dist[v] = inf for all v ≠ s
for each edge (u, v) ∈ E, in order of increasing dist[u]:
    if dist[u] + w(u, v) < dist[v]:  dist[v] = dist[u] + w(u, v);  prev[v] = u
```

Backtracking through prev[] from t to s gives the sequence of calibrated legs to run, in order. Each leg is executed exactly as a single route is today, as an independent, directly calibrated movement, with the current location updating automatically at every intermediate stop.

For a fixed room layout, all-pairs shortest paths can instead be pre-computed once with the Floyd–Warshall algorithm [8], so any later lookup is a table read regardless of hop count:

```
dist[i][j] = w(i, j)  if edge (i, j) exists, else inf;  dist[i][i] = 0
```

```
    for k in V:  for i in V:  for j in V:
        dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

The run time is $O(|V|^3)$, which is trivial at the scale of a house (tens of rooms, not thousands), and it only needs recomputing when the edge set changes, that is, when a room is added or a leg is recalibrated. This is the main lever on the calibration cost described in Section 5.4: instead of the roughly N^2 direct measurements that full connectivity requires, only the real adjacency edges of the house need measuring, and the graph search supplies everything else.